Docker Container Monitoring & Management Dashboard

End-to-end DevOps project guide — Amazon Linux / EC2 / Docker / Node.js / Express / Dockerode

Recommended project name: Docker Container Monitoring & Management Dashboard

Short project name: DockerOps Dashboard

Goal: Build a web dashboard that connects to the Docker Engine and lets a user monitor containers, view CPU/memory information, inspect logs, list images, and perform container lifecycle operations.

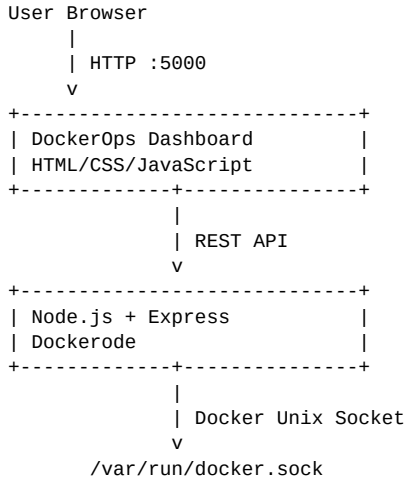
1. Project Overview

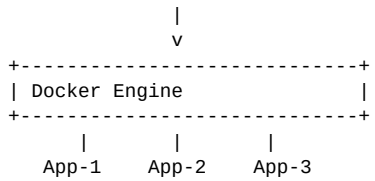
This project demonstrates practical DevOps skills by building a small internal platform for Docker operations. The browser communicates with a Node.js/Express REST API. The API uses Dockerode to communicate with the Docker Engine through `/var/run/docker.sock`. The dashboard automatically refreshes Docker information every 10 seconds.

2. Technologies Used

Layer	Technology
Cloud/Server	AWS EC2
OS	Amazon Linux
Container Platform	Docker
Container Orchestration for the app	Docker Compose
Backend	Node.js + Express
Docker API client	Dockerode
Frontend	HTML + CSS + JavaScript
Version Control	Git + GitHub

3. Architecture





4. Features

• Total container count

• Running and stopped container count

• Docker Engine version

• Docker image list

• Container start

• Container stop

• Container restart

• Container deletion

• Container logs

• CPU and memory monitoring can be extended through the Docker stats API

• Automatic dashboard refresh

• Docker health/API connectivity check

5. Step-by-Step: Prepare the EC2 Server

Launch an EC2 instance using Amazon Linux. Connect using SSH and verify the architecture.

```
uname -m
cat /etc/os-release
```

For a normal x86_64 instance, the first command should show `x86_64`. ARM instances normally show `aarch64`.

6. Install/Verify Docker

```
sudo dnf update -y
sudo dnf install -y docker
sudo systemctl enable --now docker

docker --version
sudo systemctl status docker --no-pager
```

If your Amazon Linux image already has Docker installed, you can skip the installation and only verify the service.

7. Fix the Buildx 'exec format error'

If you see `docker-buildx: exec format error`, the installed Buildx executable is usually incompatible with the EC2 architecture or corrupted. Do not keep rerunning the application build until this is fixed.

```
uname -m
docker --version
docker compose version
docker buildx version
file /usr/local/lib/docker/cli-plugins/docker-buildx 2>/dev/null || true
```

If the broken file is present at the path shown in your error, remove that stale plugin:

```
sudo rm -f /usr/local/lib/docker/cli-plugins/docker-buildx
sudo rm -f /usr/libexec/docker/cli-plugins/docker-buildx
hash -r
```

Then verify the Docker installation again. If `docker compose version` works but Buildx is missing, install the matching Buildx package/binary for your exact architecture rather than copying an x86_64 binary onto an ARM host. A good troubleshooting rule is: **architecture first** → **Docker version** → **Compose** → **Buildx**.

8. Create the Project

```
mkdir -p ~/docker-dashboard/backend/public
cd ~/docker-dashboard

tree
# Expected:
# backend/
#   public/
# docker-compose.yml
```

9. Backend

Create `backend/package.json`, `backend/server.js`, and `backend/Dockerfile`. The backend exposes REST endpoints such as `/api/info`, `/api/containers`, `/api/images`, and container lifecycle endpoints.

Important backend dependency list:

```
express
dockerode
cors
```

10. Frontend

The frontend is a self-contained HTML page containing the dashboard UI and JavaScript calls to the backend REST API. It displays containers, images, status, actions and logs. The page refreshes periodically so the dashboard reflects Docker state.

11. Docker Compose

```
services:
  dashboard:
    build:
      context: ./backend
    container_name: docker-dashboard
    ports:
      - "5000:5000"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    restart: unless-stopped
```

The Docker socket mount is what allows the dashboard container to communicate with the host Docker Engine. This is powerful and should be treated as privileged access.

12. Build and Run

```
cd ~/docker-dashboard

docker compose config

docker compose up -d --build

docker ps
docker compose ps
docker compose logs -f
```

13. Access the Dashboard

```
hostname -I

# Then open:
# http://YOUR_EC2_PUBLIC_IP:5000
```

In the EC2 Security Group, allow inbound TCP port 5000 only from your own IP for a safer lab setup. Avoid opening the dashboard to the whole internet unless you add authentication and HTTPS.

14. Test the Project

Docker health

```
curl http://localhost:5000/api/health
```

Dashboard info

```
curl http://localhost:5000/api/info
```

Container API

```
curl http://localhost:5000/api/containers
```

Images API

```
curl http://localhost:5000/api/images
```

Running containers

```
docker ps
```

All containers

```
docker ps -a
```

15. GitHub Workflow

```
git init
git add .
git commit -m "Initial Docker container dashboard"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/docker-container-dashboard.git
git push -u origin main
```

Use a clear repository README with architecture, features, setup steps, screenshots and security notes.

16. Security Improvements for a Production Version

• Add login/authentication and role-based access control.

• Put the dashboard behind Nginx or an AWS Application Load Balancer with HTTPS.

• Restrict the EC2 Security Group to trusted source IPs.

• Avoid exposing the Docker socket directly to untrusted users.

• Use a Docker socket proxy or a carefully scoped management service.

• Add audit logs for start/stop/restart/delete operations.

• Add rate limiting and input validation.

• Use monitoring/alerting for host and container failures.

17. LinkedIn Post — Ready to Publish

```
■ Built a Docker Container Monitoring & Management Dashboard

I recently built a hands-on DevOps project to monitor and manage Docker containers through a web-based dashboard.

■ Tech Stack:
• AWS EC2
• Amazon Linux
• Docker
• Docker Compose
• Node.js
• Express.js
• Dockerode
• HTML, CSS & JavaScript
• Git & GitHub
```

■ Dashboard features:
■ View running/stopped containers
■ Start, stop and restart containers
■ Delete containers
■ View container logs
■ View Docker images
■ Display Docker version
■ Docker health/API connectivity
■ Automatic dashboard refresh

■■ Architecture:
Browser → Dashboard UI → Node.js/Express API → Dockerode → Docker Engine

This project helped me understand how applications can interact with the Docker Engine programmatically and how DevO

■ I also learned an important security consideration: access to /var/run/docker.sock is highly privileged, so a prod

Next improvements:
■ CPU & memory charts
■ Authentication
■ Prometheus/Grafana monitoring
■ HTTPS
■ Container health alerts

#DevOps #Docker #AWS #CloudComputing #NodeJS #DockerCompose #Linux #DevOpsProjects #AWSCloud #GitHub

18. Portfolio / Resume Project Entry

```
Docker Container Monitoring & Management Dashboard
Tech: Docker, Docker Compose, Node.js, Express.js, Dockerode, AWS EC2, Linux, HTML/CSS/JavaScript

• Developed a web-based dashboard to monitor and manage Docker containers through the Docker Engine API.
• Implemented container lifecycle operations including start, stop, restart and delete.
• Added Docker image listing, container log viewing, Docker version/health information and automatic refresh.
• Containerized the dashboard backend with Docker Compose and integrated it with the host Docker Engine through the
• Deployed and tested the application on an AWS EC2 Linux environment.
```

19. Interview Explanation

How do you explain this project?

I built a Docker management dashboard that provides a browser-based interface for common Docker operations. The frontend sends REST requests to a Node.js/Express backend. The backend uses Dockerode to communicate with the Docker Engine through the Unix socket. The dashboard can list containers and images, show container status and logs, and perform start, stop, restart and delete operations. I containerized the dashboard itself with Docker Compose and deployed it on an AWS EC2 Linux server.

20. Troubleshooting Checklist

• If Buildx says 'exec format error': check uname -m and remove the wrong/stale Buildx binary.

• If Docker is not running: sudo systemctl enable --now docker.

• If Docker permission is denied: check Docker socket permissions and whether your user is in the docker group.

• If port 5000 is unreachable: check the EC2 Security Group and whether the dashboard container is running.

• If the Compose build fails: run docker compose config and docker compose logs.

• If the dashboard cannot access Docker: verify /var/run/docker.sock is mounted into the dashboard container.

21. Final Project Checklist

■ EC2 instance created

■ Docker installed and running

■ Docker Compose working

■ Buildx architecture verified

■ Project files created

■ Backend API working

■ Frontend dashboard working

■ Docker Compose build successful

■ Port 5000 restricted appropriately

■ GitHub repository created

■ README added

■ LinkedIn post published

■ Project added to portfolio/resume

Suggested GitHub repository name: docker-container-monitoring-dashboard

Suggested portfolio category: DevOps / Docker / AWS